

Radix-4

DP8 (8×4) — one digit per pair of bits of B

Step 1 — the dot product

$$\sum_{i=0}^7 A_i B_i$$

Step 2 — group the bits in pairs and exchange the sums

Each B_i is a 4-bit value read as signed; each pair of bits is one digit and the top pair holds the negative bit, $B_i = d_i^0 + 4 d_i^1$ with

$$d_i^0 = 2 B_i^1 + B_i^0 \in \{0, 1, 2, 3\} \quad d_i^1 = -2 B_i^3 + B_i^2 \in \{-2, -1, 0, 1\}$$

$$\sum_{i=0}^7 A_i B_i = \sum_{i=0}^7 \left[\sum_{j=0}^1 4^j d_i^j \right] A_i = \sum_{j=0}^1 4^j \sum_{i=0}^7 d_i^j A_i$$

When B_i is a lower slice of a wider B the top pair is read unsigned, $d_i^1 \in \{0, 1, 2, 3\}$.

Step 3 — the partial products

$$\sum_{i=0}^7 A_i B_i = \sum_{i=0}^7 \underbrace{\left[d_i^0 A_i \right]}_{R_0(B_i^1, B_i^0)} + 4 \sum_{i=0}^7 \underbrace{\left[d_i^1 A_i \right]}_{R_1(B_i^3, B_i^2)}$$

Each takes one of four values; the digit 3 is the cost, $3A_i = 2A_i + A_i$ needs an adder:

B_i^1	B_i^0	d_i^0	R_0	B_i^3	B_i^2	d_i^1	R_1
0	0	0	0	0	0	0	0
0	1	1	A_i	0	1	+1	A_i
1	0	2	$2A_i$	1	0	-2	$-2A_i$
1	1	3	$3A_i$	1	1	-1	$-A_i$

Step 4 — split the work

There is nothing to encode: the digit is the bit pair itself, and the dispatcher sends it to the PE as the control signals of the selection; only the selection stays inside:

$$\underbrace{\{B_i^1, B_i^0\} \longrightarrow d_i^0 \quad \{B_i^3, B_i^2\} \longrightarrow d_i^1}_{\text{dispatcher: encoding}} \longrightarrow \underbrace{\sum_{i=0}^7 d_i^0 A_i + 4 \sum_{i=0}^7 d_i^1 A_i}_{\text{PE: selection}}$$

Each d_i^j travels as its bit pair, which the cell turns into one of four multiples: 4 control bits per lane. The multiple $3A_i$ is either an adder in every cell or a third value precomputed once per row, as Bit-Plane does with its pair sums.

B is the narrow operand: 2 digits × 8 lanes = 16 selections of 10 bits per DP8. Booth and NR4SD are two ways of removing the digit 3.